IBM Innovation Center for Education

Savitribai Phule Pune University

PHEMESOFT Software That Matters

TRUSTED DELIVERY PARTNER for IBM ICE

# Department of Technology, SPPU

# Deepfake Face Detection Using Deep Learning

## PROJECT SYNOPSIS

Deepfake Face Detection using Deep Learning

**BACHELOR OF TECHNOLOGY**
Data Science

SUBMITTED BY

Vivek Deshmukh - Roll No. BT24DSL38

GUIDED BY

Prof. Manisha Bharti

# TITLE PAGE

| | |
|---|---|
| Name of Student | Vivek Deshmukh |
| Class Roll No. | BT24DSL38 |
| Email | vivek162005@gmail.com |
| Branch | Data Science |
| Proposed Topic | Deepfake Face Detection Using Deep Learning |
| Guide Name | Prof. Manisha Bharti |

# TABLE OF CONTENTS

## 1. Introduction

Deepfakes are media that have been generated or altered by AI, especially images and videos of faces, which seem very authentic but are artificially created. With rapid advances in Generative Adversarial Networks (GANs) and deep learning, the creation of convincing fake faces has become increasingly accessible, raising serious concerns around misinformation, identity fraud, impersonation, and digital trust.

This project presents a deep learning-based system for binary classification of face images as real or fake. The system is built using PyTorch with a transfer-learned ResNet18 backbone and deployed via a browser-accessible Gradio interface. To enhance transparency and explainability, Gradient-weighted Class Activation Mapping (Grad-CAM) is integrated to visually highlight the image regions that most influence each prediction.

The key technologies and techniques used in this project include:

- PyTorch — deep learning framework for model building and training
- ResNet18 — pretrained convolutional neural network used as a feature extractor
- Transfer Learning — frozen backbone strategy for stable and efficient training
- Grad-CAM — gradient-based visual interpretability method
- Gradio — lightweight Python library for building and deploying ML web interfaces
- Jupyter Notebook — interactive environment for training pipeline and analytics

## 2. Literature Survey

Deepfake detection has been an active area of research since the term gained prominence around 2017–2018, coinciding with the rise of GAN-based face synthesis tools.

### 2.1 GAN-Based Face Generation

Karras et al. (2019) introduced StyleGAN, which generates photorealistic human faces that are indistinguishable from real photographs to the naked eye. The fake images used in this project's dataset are sourced from StyleGAN outputs, making them a representative and challenging benchmark.

### 2.2 CNN-Based Detection Approaches

Rossler et al. (2019) published FaceForensics++, a large-scale dataset for facial manipulation detection, and showed that CNN-based classifiers achieve strong performance on binary real-vs-fake tasks. ResNet architectures in particular have proven highly effective in this domain due to their residual connection design, which allows deep networks to be trained stably.

### 2.3 Transfer Learning for Detection

Tolosana et al. (2020) reviewed deepfake detection methods and highlighted that transfer learning from ImageNet-pretrained models significantly reduces training time while maintaining competitive accuracy, particularly when labeled deepfake data is limited or expensive to collect.

### 2.4 Explainability in Detection Systems

Selvaraju et al. (2017) proposed Grad-CAM as a method to produce visual explanations for CNN decisions. In the context of deepfake detection, Grad-CAM helps verify whether the model is focusing on semantically meaningful face regions (e.g., eyes, skin texture) rather than image artifacts or background noise.

## 3. Methodology / Planning of Work

The project follows a structured pipeline from dataset preparation to deployment.

### 3.1 Dataset Preparation

The model uses the 140K Real and Fake Faces dataset from Kaggle, consisting of 70,000 real images (from NVIDIA's Flickr Faces HQ) and 70,000 StyleGAN-generated fake images, all resized to 256×256 pixels. The dataset is already divided into training, validation, and test sets, with CSV files provided for easy access.

### 3.2 Preprocessing and Augmentation

Training images are augmented using random resized crops, horizontal flips, and slight color jittering to improve generalization. All images are normalized using ImageNet mean and standard deviation. For evaluation, images are only resized and normalized to ensure consistent results.

### 3.3 Model Architecture

A pretrained ResNet18 is used as the backbone with frozen weights. The original classification layer is replaced by a custom head consisting of fully connected layers (input → 256 → 64 → 2) with ReLU activations and dropout. Only this head is trained, reducing computational cost and overfitting.

### 3.4 Training Configuration

The model is trained using the AdamW optimizer (learning rate $3\times10^{-4}$, weight decay $1\times10^{-4}$) and CrossEntropyLoss with label smoothing (0.05). A ReduceLROnPlateau scheduler is applied. Training is performed with a batch size of 32 for up to 10 epochs, with early stopping (patience = 3). The input image size is 224×224, and a seed value of 42 ensures reproducibility.

### 3.5 Evaluation

Performance is evaluated using loss, accuracy, F1-score, ROC-AUC, and a confusion matrix. The best model checkpoint is saved based on validation performance.

### 3.6 Deployment

The trained model is deployed using a Gradio web application. Users can upload an image and receive a real/fake prediction with confidence, along with Grad-CAM visualizations and performance metrics such as confusion matrix, ROC curve, and training history.

## 4. Facilities Required for Proposed Work

### 4.1 Software Requirements

- Python 3.x
- PyTorch & Torchvision — model building and training
- Gradio — web-based deployment interface
- NumPy & Matplotlib — numerical computation and visualisation
- PIL / Pillow — image loading and preprocessing
- Jupyter Notebook — interactive training and analytics environment
- tqdm — training progress tracking

### 4.2 Hardware Requirements

- GPU-enabled system recommended for training
- Minimum 8 GB RAM; 16 GB recommended
- Sufficient disk storage for dataset and model artifacts
- Standard CPU system sufficient for inference/deployment only